

CREATE TABLE con claves primarias y restricciones

Ya sabes qué tipo de dato le corresponde a cada columna. Ahora viene el momento de escribir la instrucción que convierte ese diseño en una tabla real dentro de la base de datos. Y hay una diferencia importante entre crear una tabla correctamente y simplemente crear una tabla: las restricciones.

Una tabla sin restricciones funciona, pero se comporta como una hoja de cálculo escondida dentro de un motor de base de datos. Cualquier dato entra, no importa si tiene sentido o no. Las restricciones son las que hacen que tu base de datos sea confiable: le dicen al motor exactamente qué puede entrar en cada columna y qué no.

En esta clase seguimos practicando en OneCompiler. Todavía no estamos construyendo la base de datos definitiva de Tienda Latam, sino aprendiendo los comandos para que cuando lleguemos a ese punto no cometamos errores.

Crear una tabla: la estructura básica

El comando para crear una tabla es `CREATE TABLE`. Después del nombre de la tabla, abres paréntesis y dentro defines cada columna con su nombre y su tipo de dato, separados por comas.

SQL

```
CREATE TABLE demo_tipos (  
    id            INTEGER,  
    nombre        TEXT,  
    precio        NUMERIC(10,2),  
    activo        BOOLEAN,  
    fecha_ingreso DATE,  
    creado_en     TIMESTAMP
```

```
);
```

Cada línea dentro del paréntesis es una columna. Primero el nombre, luego el tipo de dato. Nada más.

Dos reglas de sintaxis que no cambian:

- Cada columna va separada de la siguiente por una **coma**.
- La última columna **no lleva coma** al final.
- Toda instrucción SQL termina con **punto y coma**. Sin él, el motor sigue leyendo y puede generar errores.

Para ver que la tabla se creó y que los datos entran bien, puedes insertar una fila de prueba y luego consultarla:

SQL

```
INSERT INTO demo_tipos VALUES (1, 'Producto demo', 29.99,
TRUE, '2024-01-15', NOW());

SELECT * FROM demo_tipos;
```

INSERT INTO es el comando para agregar datos. **VALUES** lleva los valores en el mismo orden en que definiste las columnas. **SELECT *** muestra todo lo que hay en la tabla.

Esto funciona, pero tiene un problema: no hay nada que le impida a alguien insertar una fila sin ID, con dos productos con el mismo correo, o con un precio vacío. Para eso existen las restricciones.

Llaves primarias: dos formas de definirlas

Una tabla sin llave primaria no puede garantizar que sus registros sean únicos. Ya vimos en clases anteriores para qué sirve la llave primaria; ahora toca escribirla en código.

Forma 1: SERIAL

SERIAL es un tipo de dato especial que hace dos cosas a la vez: define el campo como número entero y lo incrementa automáticamente cada vez que se inserta un registro. El primer registro recibe el ID 1, el segundo el ID 2, y así sucesivamente.

sql

```
SQL
CREATE TABLE ejemplo_serial (
    id      SERIAL PRIMARY KEY,
    nombre  TEXT NOT NULL
);
```

No tienes que indicar el ID al insertar datos: el motor lo asigna solo.

Forma 2: GENERATED ALWAYS AS IDENTITY

Es el estándar más moderno y el recomendado en versiones actuales de PostgreSQL. Hace lo mismo que SERIAL pero de forma más explícita y controlada.

sql

```
SQL
CREATE TABLE ejemplo_identity (
    id      INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY
    KEY,
    nombre  TEXT NOT NULL
);
```

La sintaxis es más larga, pero deja claro exactamente qué está haciendo cada parte: el campo es un entero, siempre se genera automáticamente como identidad, y es la llave primaria.

Ambas formas logran el mismo resultado. En este curso vas a ver las dos.

Restricciones de campo: NOT NULL, UNIQUE y DEFAULT

Las restricciones son reglas que el motor aplica cada vez que alguien intenta insertar o modificar datos. Si un dato no cumple la regla, el motor lo rechaza y devuelve un error. Eso es exactamente lo que quieres.

NOT NULL — el campo no puede quedar vacío

sql

```
SQL
nombre TEXT NOT NULL
```

Si alguien intenta insertar un registro sin nombre, el motor no lo acepta. Se usa en campos que son obligatorios por lógica del negocio: un cliente siempre tiene nombre, un producto siempre tiene precio.

UNIQUE — el valor no puede repetirse en la tabla

sql

```
SQL
correo VARCHAR(100) NOT NULL UNIQUE
```

Garantiza que no haya dos registros con el mismo valor en ese campo. Perfecto para correos electrónicos, números de documento o cualquier identificador externo que debe ser único.

NOT NULL y **UNIQUE** se pueden combinar en la misma columna, como en el ejemplo de arriba.

DEFAULT — valor automático cuando no se especifica uno

sql

```
SQL
fecha_registro  DATE          NOT NULL DEFAULT CURRENT_DATE,
activo          BOOLEAN       NOT NULL DEFAULT TRUE
```

DEFAULT define qué valor toma el campo si no se especifica uno al insertar. En el ejemplo:

- **fecha_registro** toma la fecha del sistema automáticamente.
- **activo** empieza en **TRUE** porque todo cliente nuevo está activo por defecto.

Si en algún momento hay que desactivar un cliente, se actualiza ese valor. Pero al crear el registro no hace falta escribirlo: el motor lo pone solo.

Todo junto: la tabla de clientes

Aquí está la tabla de clientes de Tienda Latam con todas las restricciones aplicadas:

```
SQL
CREATE TABLE clientes (
    cliente_id      INTEGER      GENERATED ALWAYS AS IDENTITY
    PRIMARY KEY,
    nombre          TEXT         NOT NULL,
    correo          VARCHAR(100) NOT NULL UNIQUE,
    telefono        VARCHAR(20),
    fecha_registro  DATE         NOT NULL DEFAULT
    CURRENT_DATE,
    activo          BOOLEAN      NOT NULL DEFAULT TRUE,
    tipo_cliente_id INTEGER      NOT NULL
);
```

Vale la pena leer esta tabla columna por columna y entender cada decisión:

- **cliente_id** se genera automáticamente. No hay que ingresarlo.
- **nombre** es obligatorio. Un cliente sin nombre no existe.
- **correo** es obligatorio y único. No pueden haber dos clientes con el mismo correo.

- **telefono** no tiene restricciones. Hay clientes que no tienen teléfono y eso está bien.
- **fecha_registro** se llena sola con la fecha del sistema. No hay que escribirla.
- **activo** empieza en **TRUE**. Si hay que darlo de baja, se actualiza después.
- **tipo_cliente_id** es obligatorio y va a conectar esta tabla con otra más adelante.

Cada restricción responde a una regla del negocio. No son caprichos técnicos: reflejan cómo funciona Tienda Latam en la realidad.

Cómo verificar que la tabla quedó bien

Una vez creadas las tablas, puedes consultarle al motor exactamente cómo quedaron definidas: nombres de columnas, tipos de datos, restricciones y valores por defecto.

sql

```
SQL
SELECT table_name, column_name, data_type, is_nullable,
column_default

FROM information_schema.columns

WHERE table_schema = 'public'

ORDER BY table_name;
```

Esta consulta devuelve la estructura de todas las tablas que creaste. Es útil para verificar que las restricciones se aplicaron correctamente antes de empezar a insertar datos reales.

La diferencia real entre una tabla con y sin restricciones

Sin restricciones	Con restricciones
Cualquier dato entra	Solo entran datos válidos
Los errores aparecen en el análisis	Los errores se detectan al insertar
Dos clientes pueden tener el mismo correo	El motor lo rechaza automáticamente
La fecha de registro puede quedar vacía	Se llena sola con la fecha del sistema
Se parece a una hoja de cálculo	Se comporta como una base de datos real

Con las tablas creadas y las restricciones definidas, el siguiente paso es empezar a poblarlas con datos reales y aprender a consultarlos. Eso requiere dominar los comandos de inserción y consulta, que son los que más vas a usar en el día a día.